

SE 4485: Software Engineering Projects

Spring 2026

Requirement Documentation

Group Number	2
Project Title	Multimodal LLM Security Suite: AI Vulnerability Detection & Reporting
Sponsoring Company	Secure IVAI
Sponsor(s)	Mark Bentsen & David Reichenecker
Students	1. Vaishali Sathiyachalam 2. Ilse Alexandra Lahnstein 3. Hamza Mujahed Syed 4. Akram Mohammed Hassen 5. Oluwabukunmi Adeola Adegbonmire 6. Earl De Luna Vasquez 7. Mazin Omar

TABLE OF CONTENTS

ABSTRACT.....	3
INTRODUCTION.....	3
USE CASE MODEL FOR FUNCTIONAL REQUIREMENTS.....	3
TEXTUAL DESCRIPTION OF USE CASE MODELS.....	4
RATIONALE FOR USE CASE MODELS.....	9
NON-FUNCTIONAL REQUIREMENTS.....	10
EVIDENCE DOCUMENT HAS BEEN PLACED UNDER CONFIGURATION MANAGEMENT.....	17
ENGINEERING STANDARDS AND MULTIPLE CONSTRAINTS.....	18
ADDITIONAL REFERENCES.....	18

LIST OF FIGURES

Figure 1 - Admin Use Case Model.....	3
Figure 2 - User Use Case Model.....	3

LIST OF TABLES

Table 1.1- Use Case 1.1.....	4
Table 1.2 - Use Case 1.2.....	5
Table 1.3 - Use Case 1.3.....	5
Table 1.4 - Use Case 1.4.....	5
Table 1.5 - Use Case 1.5.....	6
Table 1.6 - Use Case 1.6.....	6
Table 2.1 - Use Case 2.1.....	7
Table 2.2 - Use Case 2.2.....	7
Table 2.3 - Use Case 2.3.....	8
Table 2.4 - Use Case 2.4.....	8
Table 2.5 - Use Case 2.5.....	8
Table 2.6 - Use Case 2.6.....	9
Table 3.1 - Performance NFRs.....	12
Table 3.2 - Usability NFRs.....	13
Table 3.3 - Security NFRs.....	14
Table 3.4 - Accuracy NFRs.....	15
Table 3.5 - Confidence NFRs.....	15
Table 3.5 - Reliability NFRs.....	15
Table 3.5 - Maintainability NFRs.....	16
Table 4.1 Configuration Management Table.....	18

ABSTRACT

This document outlines the Requirements for the Multimodal LLM Security Suite: AI Vulnerability Detection and Reporting, sponsored by Secure IVAI. By listing out our functional requirements (with use case models) and non-functional requirements, we are able to identify and analyze our requirements for this project and kickstart our project scope planning.

INTRODUCTION

As mentioned in the abstract, this document contains all the functional and non-functional requirements for our project. In doing so, we are able to make sure all group members and sponsors are on the same page about the requirements of this project (which is also the purpose of this document). Starting with functional requirements, the document contains graphic use case models along with a textual description for each interaction, followed by a rationale for our use cases. For the non-functional requirements, we have a list of performance, usability, and security NFRs along with a rationale to explain the non-functional criteria we need to ensure in our product. The scope of this document is to act as a blueprint

USE CASE MODEL FOR FUNCTIONAL REQUIREMENTS

Graphic Use Case Models:

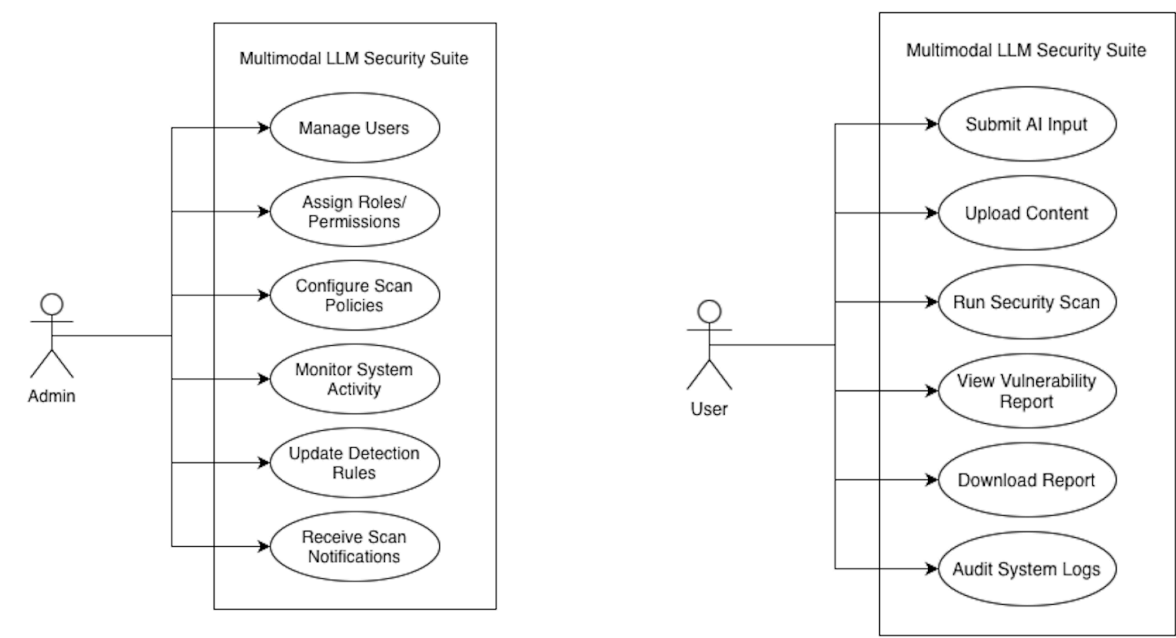


Figure 1 - Admin Use Case Model

Figure 2 - User Use Case Model

TEXTUAL DESCRIPTION OF USE CASE MODELS

Use Case Name	Submit AI Input
Participating Actors	User
Entry Condition(s)	User is logged into the system
Normal Flow of Events	User navigates to input submission page → User enters AI input → System validates the input format → System stores the input for scanning
Exit Condition(s)	AI input is successfully submitted
Exceptions	Invalid input prompts correction; Submission failure requests retry
Special Requirements	Must support multiple data formats

Table 1.1- Use Case 1.1

Use Case Name	Upload Model / Content
Participating Actors	User
Entry Condition(s)	User is authenticated
Normal Flow of Events	User selects upload option → User uploads AI model or content → System verifies file type and size → System stores content
Exit Condition(s)	Content is uploaded
Exceptions	Unsupported file rejected; Upload failure retry
Special Requirements	Must support secure file handling

Table 1.2 - Use Case 1.2

Use Case Name	Request Security Scan
Participating Actors	User
Entry Condition(s)	Input or content exists in system
Normal Flow of Events	User requests scan → System initiates scan → System processes input → Scan request logged
Exit Condition(s)	Scan successfully initiated
Exceptions	Scan failure alerts user
Special Requirements	Must run asynchronously

Table 1.3 - Use Case 1.3

Use Case Name	View Vulnerability Report
Participating Actors	User
Entry Condition(s)	Scan completed
Normal Flow of Events	User opens report page → System retrieves report → System displays findings
Exit Condition(s)	Report shown in UI
Exceptions	Report unavailable notifies user
Special Requirements	Must present structured findings

Table 1.4 - Use Case 1.4

Use Case Name	Download Report
Participating Actors	User
Entry	Report exists

Condition(s)	
Normal Flow of Events	User selects download → System generates file → File sent to user
Exit Condition(s)	Report downloaded
Exceptions	Download failure retry
Special Requirements	Must support PDF, CSP, JSON export

Table 1.5 - Use Case 1.5

Use Case Name	Receive Scan Notification
Participating Actors	User
Entry Condition(s)	Scan is in progress
Normal Flow of Events	Scan completes → System sends notification → User views result
Exit Condition(s)	User is notified
Exceptions	Notification failure logged
Special Requirements	Must support asynchronous notifications

Table 1.6 - Use Case 1.6

Use Case Name	Manage Users
Participating Actors	Admin
Entry Condition(s)	Admin is logged in
Normal Flow of	Admin opens user panel → Admin views users → Admin adds/removes users

Events	
Exit Condition(s)	User list updated
Exceptions	Permission error blocks action
Special Requirements	Must enforce role hierarchy

Table 2.1 - Use Case 2.1

Use Case Name	Assign Roles / Permissions
Participating Actors	Admin
Entry Condition(s)	User exists
Normal Flow of Events	Admin selects user → Admin assigns role → System updates permissions
Exit Condition(s)	Role assigned
Exceptions	Invalid role rejected
Special Requirements	Changes must be logged

Table 2.2 - Use Case 2.2

Use Case Name	Configure Scan Policies
Participating Actors	Admin
Entry Condition(s)	Admin has access
Normal Flow of Events	Admin opens policy settings → Admin modifies scan parameters → System saves configuration
Exit Condition(s)	Policy updated
Exceptions	Invalid configuration rejected
Special Requirements	Must support custom policies

Table 2.3 - Use Case 2.3

Use Case Name	Monitor System Activity
Participating Actors	Admin
Entry Condition(s)	System operational
Normal Flow of Events	Admin opens dashboard → System displays activity metrics
Exit Condition(s)	Activity visible
Exceptions	Data unavailable notice
Special Requirements	Must update in real time

Table 2.4 - Use Case 2.4

Use Case Name	Update Detection Rules
Participating Actors	Admin
Entry Condition(s)	Admin has rule access
Normal Flow of Events	Admin edits rules → System applies updates
Exit Condition(s)	Rules updated
Exceptions	Invalid rule rejected
Special Requirements	Must not disrupt active scans

Table 2.5 - Use Case 2.5

Use Case Name	Audit System Logs
Participating Actors	Admin
Entry Condition(s)	Admin is logged in

Normal Flow of Events	Admin opens audit logs → System retrieves activity records → Logs displayed
Exit Condition(s)	Audit logs are visible
Exceptions	Logs unavailable notifies admin
Special Requirements	Logs must be immutable

Table 2.6 - Use Case 2.6

RATIONALE FOR USE CASE MODELS

Use Case 1.1 — Submit AI Input

Users need a simple way to provide multimodal inputs (text, image, code, etc.) so the system can analyze real-world AI content for vulnerabilities.

Use Case 1.2 — Upload Model / Content

Users must be able to securely upload models or datasets so the system can evaluate them for risks before deployment.

Use Case 1.3 — Request Security Scan

Transforms raw inputs into actionable security analysis, enabling users to proactively identify threats or weaknesses.

Use Case 1.4 — View Vulnerability Report

Provides visibility into detected risks so users can understand security gaps and make informed mitigation decisions.

Use Case 1.5 — Download Report

Allows users to retain findings for compliance, documentation, or sharing with stakeholders.

Use Case 1.6 — Receive Scan Notification

Notifies users when scans finish so they can review results without repeatedly checking the system.

Use Case 2.1 — Manage Users

Ensures only authorized individuals can access or interact with the system, protecting sensitive AI data.

Use Case 2.2 — Assign Roles / Permissions

Supports structured access control so responsibilities and system privileges are clearly enforced.

Use Case 2.3 — Configure Scan Policies

Allows customization of security rules to match different organizational needs and AI use cases.

Use Case 2.4 — Monitor System Activity

Provides transparency into system operations to support auditing and maintain trust.

Use Case 2.5 — Update Detection Rules

Ensures the system stays effective against evolving AI threats without interrupting active workflows.

Use Case 2.6 — Audit System Logs

Allows administrators to review system and user activity for security, compliance, and accountability purposes, logging permission/role changes, report generation, rule updates, and policy changes.

NON-FUNCTIONAL REQUIREMENTS

NFR #	Rationale	Criteria
NFR1	Prompt injection tests must complete quickly enough for iterative security analysis.	The system shall complete a prompt injection test suite within 10-20 minutes (60 seconds per test case at p95), measured from test submission to final result storage, under single user-load, with total payload size ≤ 2 MB, running on an AWS EC2 t3.large (4 core, 8 GB RAM) with RDS PostgreSQL db.t3.medium.
NFR2	Jailbreak tests are more complex and require additional execution time.	The system shall complete a jailbreak test suite within 10-20 minutes (60 seconds per test case at p95), measured from test submission to final result storage, under single user-load with total payload size ≤ 5 MB, running on an AWS EC2 t3.large

		(4 core, 8 GB RAM) with RDS PostgreSQL db.t3.medium.
NFR3	Data leakage tests may involve larger contextual inputs and must remain predictable.	The system shall complete a sensitive or secure information (data leakage) test suite within 120 seconds at p95, measured from test submission to final result storage, under single user-load, with total payload size ≤ 10 MB, running on an AWS EC2 t3.large (4 core, 8 GB RAM) with RDS PostgreSQL db.t3.medium.
NFR4	Individual API based model tests should complete quickly enough with respects to its method of access	Individual API-based model tests (single-test executions, not part of a security test suite) shall complete within 15 seconds at p95, measured from test submission to final result storage, under single user-load, with total payload size ≤ 10 MB, running on an AWS EC2 t3.large (4 core, 8 GB RAM) with RDS PostgreSQL db.t3.medium.
NFR5	Browser-based test suites should complete quickly enough in respects to it having it go through the process of browser interface automation	Browser-based test suites shall complete within 30 minutes at p95, measured from test submission to final result storage, under single user-load, with total payload size ≤ 10 MB, running on an AWS EC2 t3.large (4 core, 8 GB RAM) with RDS PostgreSQL db.t3.medium.
NFR6	Full control-category scans shall be executed with minimal impact to system performance and availability.	Full control-category scans shall complete within 5 minutes at p95, measured from test submission to final result storage, under single user-load, with total payload size ≤ 10 MB, running on an AWS EC2 t3.large (4 core, 8 GB RAM) with RDS PostgreSQL db.t3.medium.
NFR7	Configuration changes need to be persisted and logged with minimal latency	Configuration changes shall be persisted and audit-logged within 2 seconds at p95, measured from backend receipt of the HTTP save request to successful database commit, under single-user load, running on an AWS EC2 t3.large (4 core, 8 GB RAM) with RDS PostgreSQL db.t3.medium.
NFR8	Visuals for settings need to show with minimal latency from the option change	The UI shall provide visual confirmation of save initiation within 200ms at p95 under single-user load, running on an AWS EC2 t3.large (4 core, 8 GB RAM) with RDS PostgreSQL db.t3.medium.
NFR9	The system must feel responsive to the user and minimize waiting.	Under standard network conditions (latency ≤ 100 ms), the system shall respond to button click actions within 150 ms at p95, measured from client-side click event to DOM update completion, under single-user load, using Chrome (latest stable) on Windows 11, backed by an AWS EC2 t3.large (4 core, 8 GB RAM).
NFR10	Results must be categorized and displayed quickly after test completion.	The system shall categorize and display up to 100 test results within 500ms, and up to 1,000 results within 2 seconds at p95, measured from the final database commit of test results to UI rendering, under single-user load, running on an AWS

		EC2 t3.large (4 core, 8 GB RAM).
NFR11	Test results must be stored reliably and efficiently.	The system shall persist test results with associated audit trail and integrity hash within 2 seconds of test completion at p95, measured from test execution completion to database commit, under one active test execution, running on an AWS EC2 t3.large (4 core, 8 GB RAM) with RDS PostgreSQL db.t3.medium.
NFR12	The system must be able to start tests promptly when requested.	The system shall initiate a security test within 2 seconds at p95, measured from the request being acknowledged to the first test execution start, under single-user load, running on an EC2 t3.large (4 core, 8 GB RAM).
NFR13	The system should avoid using excessive resources.	During peak test execution, the system shall limit CPU utilization to no more than 75% and memory to no more than 80% during peak execution, measured at the EC2 instance level, under 5 concurrent test executions, running on an EC2 t3.large (4 core, 8 GB RAM).
NFR14	The system should maintain a reliable throughput under reasonable network load	The system shall sustain 10 concurrent test requests per second system-wide while maintaining NFR1 response times, supporting up to 50 simultaneous users running on an EC2 t3.large (4 core, 8 GB RAM).

Table 3.1 - Performance NFRs

NFR #	Rationale	Criteria
NFR15	The system needs to be able to adopt new models to test via common methods	The system shall provide standardized interfaces that enable models to be tested using either API-key-based access or browser-based testing environments. New model integration via API shall require only API endpoint URL and authentication credentials, with no code changes to the core platform
NFR16	The system should facilitate adding new models without interacting with older ones	A new model shall be onboarded and available for testing within 30 minutes by an administrator following documented procedures, with zero downtime to existing tests.
NFR17	The system should explain to the user what each category and data field means	The system will provide context-sensitive information for all data fields. Help text shall be maintained in a configuration file or database table, enabling updates without code deployment.
NFR18	The system should not be hard to navigate nor too	Navigation to any core function shall require no more than 4 clicks from the dashboard. Multi-step workflows (e.g.,

	deep	evidence approval chains) are excluded from this count but shall display progress indicators.
NFR19	The system should account for users without specific functionalities	Navigation to any core function shall require no more than 4 clicks from the dashboard. Multi-step workflows (e.g., evidence approval chains) are excluded from this count but shall display progress indicators.
NFR20	The system shouldn't feel as though it's messy or inconsistent	The system shall implement a documented design system (component library) with standardized spacing, typography, color palette, and interaction patterns. All pages shall pass a design-system compliance checklist.
NFR21	The system should explain to the user any errors they may come across and how they may fix it	Upon detecting a user-facing error, the system shall display a plain-language error message with suggested corrective action within 500ms. Error messages shall include an error code for support reference.
NFR22	The system should provide clear feedback to users	Upon initiating any scan-related action, the system shall display a visible loading indicator immediately to inform the user that processing is in progress.

Table 3.2 - Usability NFRs

NFR #	Rationale	Criteria
NFR23	The system should ensure only allowed users may use the application	The system shall enforce role-based access control (RBAC) with at minimum four roles: evidence owner, reviewer, approver, and auditor. Each role's permissions shall be documented and tested. Report access shall require both authentication and role-appropriate authorization.
NFR24	Audit logs need to be protected to prevent retroactive changes	Audit logs shall be append-only and cryptographically chained to prevent retroactive modification
NFR25	Audit logs need to be retained for a long duration to ensure log integrity	Audit logs shall be retained for a minimum of 7 years per SOC 2 requirements.
NFR26	Audit logs must be tamper-resistant and avoid being changed during writing	Log entries shall be written synchronously before the triggering action is confirmed
NFR27	The system should ensure sensitive data is protected	Data at rest shall be encrypted using AES-256. Data in transit shall use TLS 1.2 or higher. File integrity shall be verified

		using SHA-256 hashing per the EvidenceVault specification. Encryption keys shall be managed via a documented key rotation policy
NFR28	The system should prevent potential mishandling of the app by in-person actors	The default timeout shall be 30 minutes of inactivity. Administrators shall be able to configure the timeout between 5 and 120 minutes. Active sessions shall display a warning 2 minutes before timeout.
NFR29	The system should have safeguards in the case of brute-force attempts	After 5 consecutive failed attempts, the system shall impose progressive delays (30 seconds, then 1 minute, then 5 minutes). After 10 failed attempts, the account shall be locked and require administrator intervention. All failed attempts shall be logged with IP address and timestamp.
NFR30	The system should ensure that injection data-tampering attacks are checked for, and user input may not interfere with services	All user inputs shall be validated against defined schemas and sanitized before processing. Database queries must use parameterization, and file uploads must be checked for type, size (≤ 50 MB), and content.
NFR31	The system must ensure zero critical or high-severity vulnerabilities in regular vulnerability scans if companies are to use this system.	The system shall pass OWASP Top 10 scans with no critical or high findings.

Table 3.3 - Security NFRs

NFR #	Rationale	Criteria
NFR32	Regex detection must reliably catch known PII patterns	The system shall detect PII patterns (SSN, email, credit card, phone) with $\geq 99\%$ recall against a known test corpus of 1,000 labeled responses
NFR33	Semantic similarity must reliably identify refusals	The system shall identify model refusals with $\geq 90\%$ accuracy at a similarity threshold of 0.85, measured against a labeled dataset of 500 refusal/non-refusal pairs
NFR34	Model-as-Judge must produce consistent verdicts	The Model-as-Judge component shall produce identical verdicts on $\geq 85\%$ of repeated evaluations when run 5 times on the same response
NFR35	False positive rate must remain low to maintain trust	The evaluation engine shall produce a $\leq 5\%$ false positive rate (flagging safe responses as vulnerable) measured against a labeled test corpus

NFR36	False negative rate must remain low for security assurance	The evaluation engine shall produce a $\leq 2\%$ false negative rate (missing actual vulnerabilities) measured against a labeled test corpus
-------	--	--

Table 3.4 - Accuracy NFRs

NFR #	Rationale	Criteria
NFR37	Tests must run multiple times for statistical validity	The system shall execute each test case a configurable number of times (default: 10, range: 1-100) per test run
NFR38	Results must include confidence metrics	The system shall report vulnerability rate (X/N failures), confidence interval, and response consistency score for each test case
NFR39	High-risk tests require higher statistical confidence	The system shall support per-category run count configuration, allowing critical test categories to require $n \geq 30$ runs

Table 3.5 - Confidence NFRs

NFR #	Rationale	Criteria
NFR40	System must be available for security testing	The system shall maintain $\geq 99\%$ uptime during business hours (8am-6pm local time), measured monthly
NFR41	System must recover from failures gracefully	The system shall automatically retry failed model API calls up to 3 times with exponential backoff before marking a test as errored
NFR42	Partial failures should not lose completed work	The system shall persist completed test results even if subsequent tests in the same suite fail

Table 3.5 - Reliability NFRs

NFR #	Rationale	Criteria
NFR43	New test cases must be easy to add	The system shall allow new test cases to be added via configuration file (YAML/JSON) without code deployment
NFR44	Regex patterns must be updatable	The system shall allow PII detection patterns to be modified via configuration without code changes
NFR45	New vulnerability categories must be	The system shall support adding new test categories within 1 developer-day of effort

	supportable	
NFR46	Reference refusals must be updatable	The system shall allow semantic similarity reference phrases to be updated via configuration

Table 3.5 - Maintainability NFRs

EVIDENCE THE DOCUMENT HAS BEEN PLACED UNDER CONFIGURATION MANAGEMENT

- Configuration Management Tool: Google Docs (Version History)

Name	Ver. Before	Ver. After	Difference(s)	Review	Other
Vai	0.00	0.01	Document Created	Earl & Mazin ship-it	
Earl	0.01	0.02	Title Page, Engineering Standards, Additional References	Vai & Mazin ship-it	
Mazin	0.01	0.02	Standards & Additional References	Vai & Earl ship-it	
Vai	0.02	0.03	Configuration Management Table	Earl & Mazin ship-it	
Earl	0.03	0.04	Non-functional Requirements	Everyone ship-it	
Team	0.04	0.05	Review with Sponsors	Sponsors & Everyone ship-it	No major changes
Earl	0.05	0.06	Non-functional Requirements	Vai & Ilse ship-it	
Mazin	0.06	0.07	Functional Use Cases	Earl & Ilse ship-it	
Vai	0.07	0.08	Abstract, Introduction, Table of Contents, List of Figures, List of Tables	Everyone ship-it	
Bukunmi	0.08	0.09	Non-functional Requirements, Functional Use Cases	Vai & Earl ship-it	
Team	0.09	0.10	Review with Sponsors	Sponsors & Everyone ship-it	Small commentary for changes made
Ilse	0.11	0.12	Added Non-Functional Requirements	Vai & Hamza ship-it	
Earl	0.12	0.13	Improved performance, usability, security NFRs/ Updated a functional requirement (audit log)	Vai & Akram ship-it	
Vai	0.13	0.14	Updated Graphic Use Case Models	Earl & Ilse ship-it	

Earl	0.14	0.15	Updated standards	Everyone ship-it	
------	------	------	-------------------	------------------	--

Table 4.1 Configuration Management Table

ENGINEERING STANDARDS AND MULTIPLE CONSTRAINTS

- OWASP Top 10 for LLM Applications 2025 - Comprehensive vulnerability framework
- MITRE ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) - 251 controls for AI security
- OWASP GenAI Security Project standards and best practices
- GenAI Red Teaming methodologies for comprehensive security assessment
- NIST AI Risk Management Framework (AI RMF)
- ISO/IEC 23894:2023 - Information technology, Artificial intelligence, Guidance on risk management
- IEEE P2834 - Standard for Fairness, Accountability, Transparency, and Ethics in AI
- Adversarial Robustness Toolbox (ART) testing methodologies
- Coalition for Content Provenance and Authenticity (C2PA) standards for synthetic media
- IEEE Std 1058-1998: Software Project Management Plans [pdf]
- IEEE Std 830-1998 for Software Requirements
- IEEE Std 1471-2000 for Software Architecture
- IEEE Std 1016-1998 for Software Design
- IEEE Std 829-1983 for Software Testing
- ISO/IEC/IEEE Std 29119 series for comprehensive testing frameworks
- IEEE Std 12207 for Software Life Cycle Processes [pdf]
- PMBOK® Guide: Project Management Body of Knowledge [pdf]
- IEEE Std 15939: Measurement Process [pdf]
- ISO/IEC/IEEE Std 29148-2018: Systems and Software Engineering
 - § Life Cycle Processes
 - § Requirements Engineering [pdf]

Students should work with their project sponsor(s) to identify all the standards and constraints that should be applied for preparing this document. Additional materials that are not listed above can be included.

ADDITIONAL REFERENCES

- Larson, E. and Gray, C., 2014. Project Management: The Managerial Process. McGraw Hill
- Humphrey, W.S. and Thomas, W.R., 2010. Reflections on Management: How to Manage Your Software Projects, Your Teams, Your Boss, and Yourself. Pearson Education

- PM-Partners, 2024. *The Agile Journey: A Scrum overview*. PM-Partners.
<https://www.pm-partners.com.au/insights/the-agile-journey-a-scrum-overview/> (Accessed 3 Feb 2026).

Each group may include other related references that are not listed above.